CS405 Project 1 | Draft report

Transformations, Projection and Curves from Paintings
Fall 2026 · Revised 11 November 2026 · Code baseline: e0677d5 + measurement JSON viewer

Painting and scene

Kavunlu Natürmort, Şeker Ahmed Paşa (Sakıp Sabancı Museum collection) was selected for the melon slice's clear organic outline. The painting is a textured quad. Three foreground solids demonstrate transforms; a table with a rotating platform demonstrates hierarchy; curves trace the lower edge of the melon slice. The image was taken from the supplied Week 01 course material.

One Vite application uses raw WebGPU and WGSL. Mathematics, scene traversal, rendering, interaction and UI are separate modules. The common pipeline is $\mathbf{p_{clip} = P\,V\,M\,p_{object}}$. Four views share the application, reset controls and debug switch.

A. Transformations and order

Matrices use column-major storage and act on column vectors; the rightmost transform acts first. The matrix module implements identity, multiplication, translation, scaling, rotations around X/Y/Z, transpose and rigid inverse. For a rigid matrix [R,t;0,1], orthogonality gives the inverse $[R^T,-R^Tt;0,1]$. This shortcut is not valid for a scaled transform.

The middle object has XYZ translation, Y rotation and uniform scale controls. Its rotation also includes a fixed -20° X tilt: R = $R_y(32°)$ $R_x(-20°)$ in the captures. With T R S, local geometry is scaled, rotated and then translated. With S R T, the translation vector itself is rotated and scaled, changing the position even though the controls are identical. The rest of the scene uses parent-to-child composition and local T R S ordering where applicable.

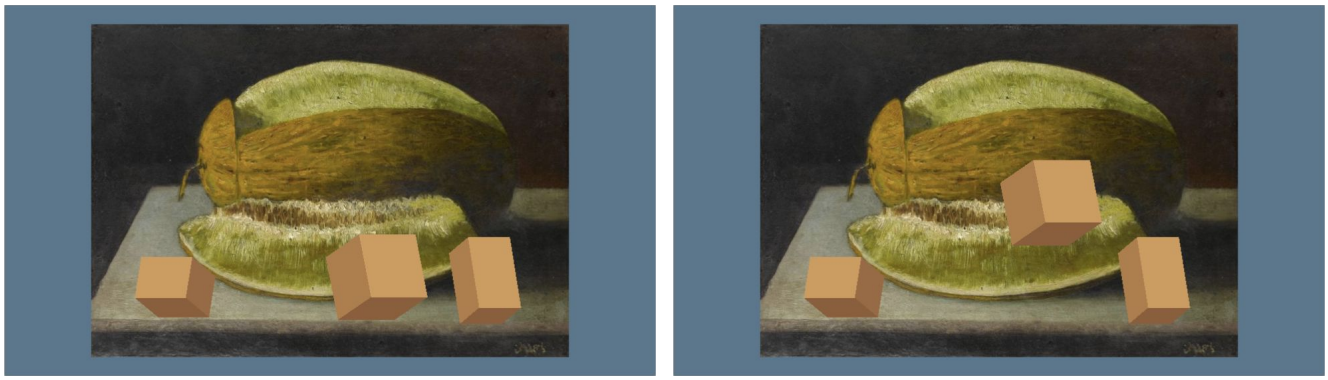


Figure 1. Canvas details from the TRS (left) and SRT (right) captures, with identical controls. Both matrices are reproduced below; the unedited full captures include the UI and matrix panels.

```
TRS (rows)                          SRT (rows)
[ 0.339 −0.072  0.199  0.280 ]      [ 0.339 −0.072  0.199  0.312 ]
[ 0.000  0.376  0.137 −0.520 ]      [ 0.000  0.376  0.137 −0.072 ]
[−0.212 −0.116  0.319  0.900 ]      [−0.212 −0.116  0.319  0.288 ]
[ 0.000  0.000  0.000  1.000 ]      [ 0.000  0.000  0.000  1.000 ]
Controls: t=(0.28,−0.52,0.90), scale=0.40, Y angle=32 deg.
```

The translation column directly verifies the origin's image because multiplying by (0,0,0,1) selects that column. A direction has homogeneous w = 0, so translation does not change it.

B. Scene graph and a leaf world matrix

Each node stores its local transform, parent, children and world matrix. Traversal computes $W_{child} = W_{parent}L_{child}$. There are nine nodes across four levels, including the root. Rotating the turntable carries both blocks without separately editing their world positions.

```
Scene
  Table
    Front left leg     Front right leg
    Back left leg      Back right leg
    Turntable
      Small block
      Tall block
```

The table and blocks have separate mesh-sizing matrices. These affect the geometry but do not scale the frame inherited by a child. The rendered model is $W_{node}S_{mesh}$; local debug axes use W_{node}. This keeps a thin tabletop from shrinking its descendants.

Numerical verification at the reset pose

For the small block, angles are in radians and the animation is paused at the reset angle 0.28. The complete chain of local frames is:

```
W = T(0,−0.28,1.03) Rx(0.32) T(0,−0.24,0)
    T(0,0.105,0) Ry(0.28) T(−0.27,0.13,0)
```

```
W_small =
  [  0.96106  0.00000  0.27636 −0.25948 ]
  [  0.08693  0.94924 −0.30232 −0.30822 ]
  [ −0.26233  0.31457  0.91227  1.09926 ]
  [  0.00000  0.00000  0.00000  1.00000 ]
```

By hand, the block origin first becomes (-0.27,0.13,0). The turntable rotation sends it to (-0.27 cos(0.28),0.13,0.27 sin(0.28)). The table and turntable translations leave y = -0.005 before the root rotation. Applying $R_x(0.32)$ and the root translation yields **(-0.25948, -0.30822, 1.09926)**, matching the last column above to rounding. Mesh scale diag(0.19,0.16,0.19,1) is applied only after this frame is obtained.

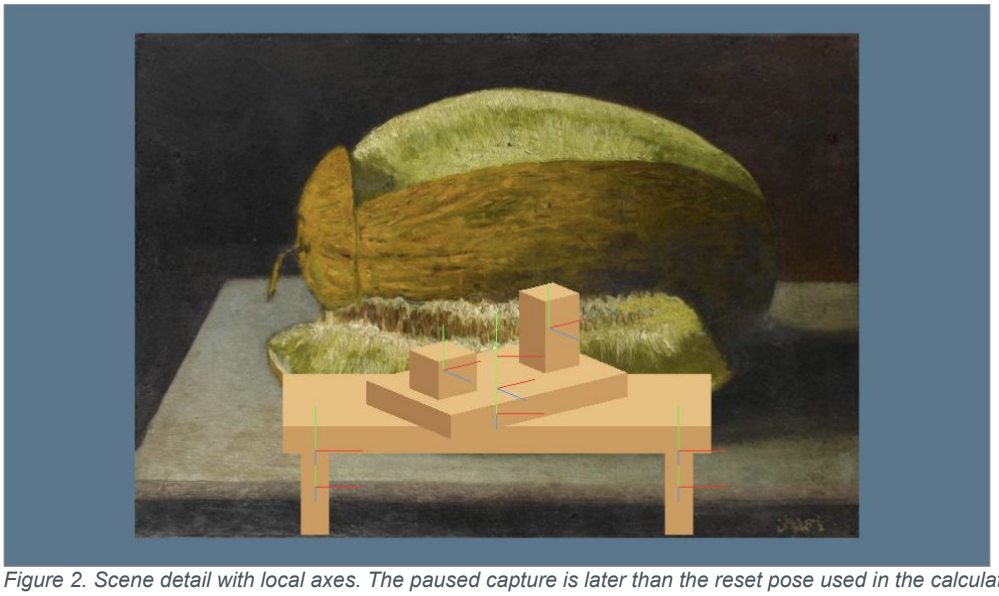


Figure 2. Scene detail with local axes. The paused capture is later than the reset pose used in the calculation above; the tree is reproduced above and in the full capture.

Reproducible verification - 11 November

All 13 Node tests passed with npm test. They cover rigid inverse, homogeneous directions, rotation order, camera degeneracy, projection/frustum corners, the leaf origin and hierarchy reset, plus curve interpolation and editing. The rotation oracle follows V3.rot in Week06/demos/w6b.js. The production build also passes; these numerical checks do not replace browser interaction tests.

C. View and projection

Deriving lookAt

For eye e, target a and reference up h, define the backward unit axis w = normalize(e - a), the right axis u = normalize(h × w), and v = w × u. Taking coordinates relative to e gives x_{view} = u·(p-e), and similarly for v and w. Therefore the rows of the view matrix are:

```
V = [ ux uy uz −dot(u,e) ]
    [ vx vy vz −dot(v,e) ]
    [ wx wy wz −dot(w,e) ]
    [  0  0  0     1     ]
```

If eye and target coincide, the implementation rejects the camera. If up is parallel to w, their cross product cannot define u; an alternate reference axis is selected before normalization. Mouse orbit updates eye around the target. The initial eye is approximately (0.45,0.61,5.04), target (0,-0.12,0.52).

WebGPU depth convention

View space looks down -Z. Let f = 1/tan(FOV_y/2), aspect r, near n and far F. The matrices below are written as rows, although stored column-major:

```
Perspective:                    Orthographic (half-height h):
[ f/r  0    0          0 ]      [1/(hr)  0     0        0 ]
[  0   f    0          0 ]      [  0    1/h    0        0 ]
[  0   0  F/(n−F)  Fn/(n−F)]    [  0     0  1/(n−F) n/(n−F)]
[  0   0   −1          0 ]      [  0     0     0        1 ]
```

At z = -n, perspective division gives z_{NDC} = 0; at z = -F it gives 1. Orthographic has w = 1 and the same endpoints. Perspective w = -z causes foreshortening; orthographic size is independent of distance. With n = 0.65 and F = 6.5, F/n = 10. Keeping near away from zero improves perspective depth precision while preserving the scene; reducing far can further limit the covered range. Precision remains nonuniform in perspective.

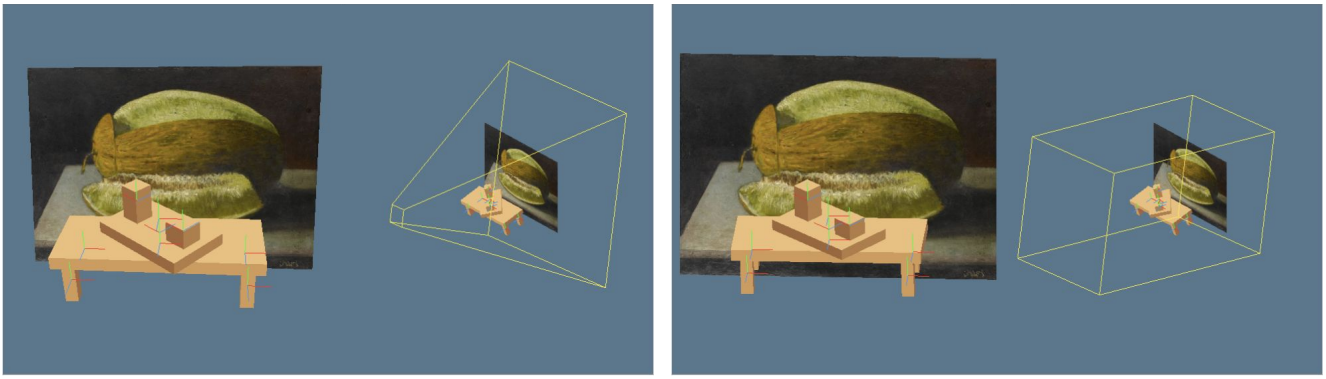


Figure 3. Canvas details: perspective (left), orthographic (right), at the same eye, target and frozen object pose. Each includes the inspection camera and yellow frustum; turntable animation is paused across the pair.

The eight near/far plane corners are transformed back to world space with V inverse and connected by twelve edges. The fixed inspection camera projects those world-space lines, exposing a perspective frustum versus an orthographic box.

Texture convention - checked against Week 07

The quad uses top-left UV (0,0), an rgba8unorm-srgb color texture, linear min/mag filtering and clamp-to-edge. Sampling decodes to linear RGB; the sRGB attachment encodes the output. This matches the Week07 texturing lecture. Only the base level exists: mipmapping and anisotropic filtering are not implemented, so minification aliasing remains possible.

D. Curves traced over the melon

Cubic basis, endpoints and de Casteljau

For a segment with control points P_0 through P_3, the Bernstein weights are $(1-t)^3$, $3(1-t)^2t$, $3(1-t)t^2$ and t^3. The curve is their weighted sum:

```
B(t) = (1−t)^3 P0 + 3(1−t)^2 t P1
     + 3(1−t)t^2 P2 + t^3 P3,     0 <= t <= 1
B'(t) = 3(1−t)^2(P1−P0) + 6(1−t)t(P2−P1)
      + 3t^2(P3−P2)
```

At t = 0 the weights become (1,0,0,0), so $B(0) = P_0$; at t = 1 they become (0,0,0,1), so $B(1) = P_3$. De Casteljau repeatedly interpolates adjacent points $Q_i = (1-t)P_i + tP_{i+1}$, reducing four points to three, then two, then one. The final point equals B(t); the intermediate levels are drawn for the active segment.

Three segments and C1 continuity

Ten control points define segments [P0..P3], [P3..P6] and [P6..P9]. Adjacent curves share an endpoint (C0). For equal segment durations, C1 requires equal endpoint derivatives: 3(P3-P2) = 3(P4-P3), hence **P4 = 2P3-P2**; likewise P7 = 2P6-P5. Initial joins satisfy P3-P2 = P4-P3 = (0.18,-0.01) and P6-P5 = P7-P6 = (0.05,0.10).

Dragging an interior handle mirrors its partner about the shared anchor; dragging an anchor also moves its neighboring handles. Moves that would put a point or its mirrored handle outside [0,1] are rejected. The global parameter assigns equal time to each segment. Animation uses the derivative direction for the red arrow; uniform parameter steps do not imply constant arc-length speed.

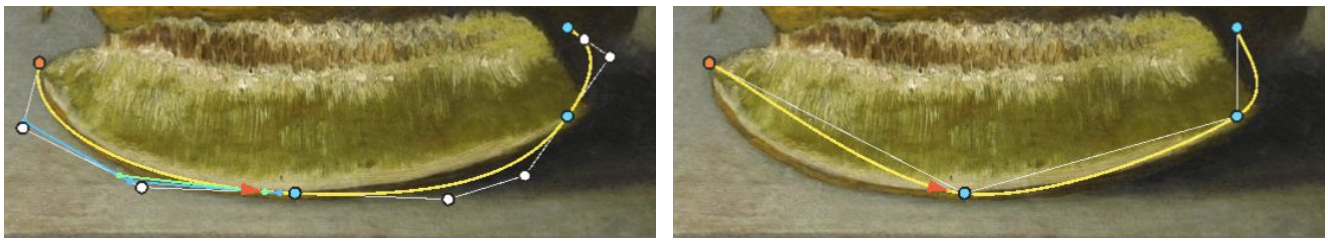


Figure 4. Enlarged melon-contour details: C1 Bézier and de Casteljau (left), Catmull-Rom (right). Global t = 0.300; the Bézier segment-local t is 0.900. Full painting captures are retained in docs/screenshots/.

Second family and export

The uniform Catmull-Rom segment interpolates the middle pair of four knots. The revised implementation adapts C6.crBez from Week06/demos/w6a.js: convert the central-difference tangents to cubic Bézier handles, then reuse the existing evaluator and derivative.

```
B0 = P1                  B1 = P1 + (P2−P0)/6
B2 = P2 − (P3−P1)/6       B3 = P2
```

Endpoint knots are repeated; the four Bézier anchors become spline knots. A 51-parameter comparison against the independent expanded polynomial passes, as do finite-difference tangent checks. The lecture examples also pass: the slide 9 cubic gives (4,3) at t=0.5; the slide 20 Catmull-Rom example gives (3,1). Both C1 joins remain valid after anchor/handle edits. Export uses bottom-left normalized coordinates; Appendix A reproduces exports/curve.json.

E. Integration, measurements and appendices

One application provides A-D views, reset, debug and recorded-JSON controls. Use npm install then npm run dev; npm run build and npm run preview serve the production version. All seven figures were refreshed on 11 November from the measured production build. Full images remain in docs/screenshots/; details are clipped here for readability.

Measured performance

Recorded 11 November on the current production build. Apple M2 (10 GPU cores), macOS 26.6.2; Codex in-app browser, reported Chromium UA 156.0.0.0. Fixed viewport 1280 × 900 CSS pixels; canvas 980 × 558, DPR 1. Each run uses a 3 s warm-up then at least 30 s foreground sampling. Animation/debug are on for B/C/D; de Casteljau renders only in Bézier mode. All intervals are retained; hiding/resizing cancels recording. Source and bundle fingerprints are in docs/measurements/build.json.

Scene	Mean ms	P95 ms	Max ms	FPS	CPU ms	Vertices
A	16.67	17.20	17.70	60.00	0.21	114
B	16.67	17.30	17.70	60.00	0.25	348
C	16.67	17.10	17.70	60.00	0.24	720
D-B	16.67	18.30	18.80	60.00	0.87	2169
D-C	16.67	18.60	18.80	60.00	0.74	1467

D-B is Bézier; D-C is Catmull-Rom. FPS = 1000/mean interval; P95 uses nearest rank. CPU time covers JavaScript rendering work through queue.submit; neither metric measures GPU execution. The maximum observed interval is 18.8 ms. Near-60 FPS pacing limits throughput conclusions. Curve modes submit different debug geometry, so their CPU averages do not isolate formula cost. One run per mode provides no repeated-run variance. Raw samples are in docs/measurements/.

Debugging and practical limitations

A stopped preview server was restarted before capture. Browser downloads were inconsistent, so a Recorded JSON section was added to expose the complete result directly. All retained runs were then repeated on that build at one fixed resolution. Source/bundle fingerprints identify the measured files. Camera screenshots now freeze the same object pose. Old measurements were removed. No GPU timestamp measurements or cross-device tests are claimed.

Appendix A. Exported curve (compact JSON)


```
{"painting":"Kavunlu Natürmort","artist":"Şeker Ahmed Paşa",
 "type":"piecewise-cubic-bezier",
 "coordinates":{"units":"normalized","origin":"bottom-left",
              "xRange":[0,1],"yRange":[0,1]},
 "segments":3,"continuity":"C1","controlPoints":[
 {"x":0.18,"y":0.40},{"x":0.16,"y":0.29},{"x":0.30,"y":0.19},
 {"x":0.48,"y":0.18},{"x":0.66,"y":0.17},{"x":0.75,"y":0.21},
 {"x":0.80,"y":0.31},{"x":0.85,"y":0.41},{"x":0.82,"y":0.44},
 {"x":0.80,"y":0.46}]}
```


Appendix B. AI assistance and sources - draft status

OpenAI Codex assisted with requirements review, planning, application/mathematics code, documentation, debugging, evidence capture, measurement tooling, tests and this report. The student stated that revised course rules allow this assistance. The student-maintained detailed usage record still needs final review. The Catmull-Rom helper is adapted from the course source, as credited on page 4.

Sources: CS405_Project1_2026.docx; Week01 painting asset; Week06/L1-Mon-Nov02-Curves.html (slides 9, 11, 16, 20), demos/w6a.js (C6.crBez) and demos/w6b.js (V3.rot); Week07/L1-Mon-Nov09-Texturing.html (Level 10). Detailed attribution: docs/course-material-review.md. Repository: github.com/efe-akkus/cs405-project1. This remains a review draft.